Name:

BUID:

Workshop 5: Final Exam Review

1. Implement `val count : 'a -> 'a list -> int` so that `count x l` is the number of occurrences of `x` in `l`.
2. Implement the function from the previous problem with a single call to `fold_left`.
3. Design an ADT `type tree23` which can be used to represent trees in which every node either has two or three children.
4. Implement the function `val list_of_tree23 : 'a tree23 -> 'a list` which converts a `'a tree23` into an `'a list` by listing the elements in order.
5. Consider the following OCaml program.

```
type 'a foo = Foo of ('a foo -> 'a)
let bar (Foo f) = f
let baz x = bar x x
let out = baz ???
```

   Given an expression to put in place of `???` such that `out` evaluates to `2`.

6. Consider the following OCaml program.

```
let x y =
  let x = y in
  let y x = x + 1 in
  let x y = y x in
  x y
```

   (a) What is the the type of `x`?

   (b) What is the value of `x 2`?

7.

$$\begin{aligned}
\langle a \rangle &::= \langle b \rangle \langle c \rangle \\
\langle b \rangle &::= \langle b \rangle \texttt{y} \mid \texttt{x} \\
\langle c \rangle &::= \langle d \rangle \mid \langle d \rangle \texttt{y} \\
\langle d \rangle &::= \texttt{z} \langle b \rangle \mid \texttt{z}
\end{aligned}$$

   (a) Draw a parse tree for `x y z`

   (b) Determine the shortest sentence accepted by this grammar that has multiple parse trees

8. In OCaml, there is an operators called `;` which can be used to sequence expressions.[1] Consider the following toy grammar.

$$\begin{aligned}
\langle e \rangle &::= \langle e1 \rangle \texttt{;} \langle e \rangle \mid \langle e1 \rangle \\
\langle e1 \rangle &::= \texttt{x} \mid \texttt{(} \langle e \rangle \texttt{)}
\end{aligned}$$

   (a) Draw a parse tree for `x ; ( x ; x ) ; x`

   (b) Write the BNF production rule for OCaml list literals (using <e> for the nonterminal symbol for expressions)

   (c) Demonstrate that above grammar, with this new rule, is ambiguous

9. Determine an OCaml expression with the polymorphic type `('a, 'b) result -> (('a -> 'c) * ('b -> 'c)) -> 'c`

[1] This is really only useful in the imperative fragment of OCaml.

10. Determine an OCaml expression with the polymorphic type `('a -> 'b, 'c -> 'b) result -> ('a * 'c) -> 'b` (look familiar?)

11. Consider the following expression.

```
(fun f -> fun x -> f x) (let z = 1 in fun y -> y + z)
```

    Determine its (closure) values (look familiar?). Also write a semantic derivation.

12. $$\{x : \tau_1, y : \tau_2\} \vdash \lambda z^T.xz(yz) : \top \to \top$$

    (a) Determine types τ_1 and τ_2 such that the above judgment is derivable in STLC. Also provide the derivation.

    (b) Are there multiple choices for τ_1 and τ_2? If so, give another choice and justify your answer.

13. Determine an expression in the λ-calculus that terminates using (small-step) CBN evaluation but gets stuck using (small-step) CBV evaluation.

14. Suppose we augment the rules of 320Caml so that adding a variable to a context does *not* shadow existing variables, but instead leaves the context unchanged in the case that variable is already declared. Demonstrate that this breaks preservation.

15. Determine the principal type of `fun f -> fun g -> fun x -> f (g (f x))`.

Continue your work here.

Continue your work here.